ARROW: An Adaptive Repository-Aware Repair Workflow for LLM-Based Java Unit Test Generation

Nguyen Tuan Dung[1], Vuong Kieu Anh[1], Doan Huu Minh Quang[1], Ngo Duc Chinh[1], Tran Van Han[1], Nguyen Hoang An[2], and Nguyen Thi Nguyet[1]

[1] FPT University, Hoa Lac Hi-Tech Park, Km 29 Thang Long Boulevard, Hoa Lac Commune, Hanoi 100000, Viet Nam

[2] Hanoi University of Science and Technology, 1 Dai Co Viet Road, Bach Mai Ward, Hanoi 100000, Viet Nam

E-mail: dungnthe180616@fpt.edu.vn; anhvkhe180546@fpt.edu.vn; quangdhmhe181540@fpt.edu.vn; chinhndhe180889@fpt.edu.vn; hantvhe181281@fpt.edu.vn; an.nh2413937@sis.hust.edu.vn; nguyetnt41@fpt.edu.vn

Abstract—Automated unit test generation for Java projects still faces significant challenges, as generated tests often fail to compile, do not conform to repository-specific build configurations, or introduce regressions into existing modules. This paper proposes ARROW, an adaptive repository-aware repair workflow for generating, validating, repairing, and evaluating Java unit tests using a fine-tuned open-source code model. The main contributions of this study are threefold: (1) a repository-aware generation workflow based on a fine-tuned open-source LLM that analyzes the build tool, Java version, module context, testing framework, and existing test style to construct project-specific prompts; (2) an empirical evaluation of ARROW's effectiveness using compilation success, module-test pass rate, repair success rate, code coverage, mutation score, and test smell metrics; and (3) an adaptive repair algorithm for faulty generated tests. The algorithm classifies failure states, tracks normalized error signatures and generated-code hashes, and applies checkpointing and rollback mechanisms to prevent ineffective repair loops and regressions during iterative repair. After generating a candidate test, ARROW validates it using Maven or Gradle and activates Adaptive Repair when compilation errors, execution failures, or module regressions are detected. Across 1,200 initial generations over 200 repositories and 200 class-level samples, the experiments achieved an overall compilation success rate of 70.25%. After repair, 622 tests (51.83%) reached the full valid state across all model-prompt configurations. In the controlled repair comparison, Adaptive Repair successfully repaired 56.67% of initially failed candidates with a median of 2 attempts [IQR 1–4]. Across valid generated tests, the weighted mean line coverage, branch coverage, mutation score, and smell density were 84.97%, 75.98%, 57.60%, and 0.41, respectively. The results demonstrate that combining repository-aware configuration, adaptive repair, and automated quality assessment makes LLM-based unit test generation more reproducible, easier to diagnose, and better suited to real-world Java projects.

Keywords: unit test generation; repository-aware prompting; adaptive repair; Java testing; mutation testing; large language models

I. INTRODUCTION

A. Importance of Unit Testing

Software testing is one of the important and labor-intensive phases in the software development lifecycle because it directly affects the quality, reliability, and stable operation of the system. In this context, unit test plays the role of a foundational testing layer, focusing on checking the smallest components of the source code such as class, method, or independent module. Detecting errors early at the unit level helps developers identify the causes more quickly, reduce the risk of errors spreading to integration or deployment layers, and significantly reduce the cost of fixing errors in later stages of the project [1], [5].

For object-oriented Java projects, unit testing is even more important because the behavior of the system is usually organized through class, method, constructor, field, and dependency relationships among components. A good unit test suite not only helps verify the correctness of each small function but also supports the maintenance, refactoring, and safe extension of the source code. When the system changes, unit test can act as a regression protection layer, helping detect early errors caused by modification or the addition of new features.

However, writing unit tests manually often consumes a lot of time and requires developers to deeply understand the processing logic, source code structure, and internal dependencies of the project. As software scale increases, the number of class, method, and test scenarios also increases, making the process of creating and maintaining unit tests more complex. Therefore, automating the unit test generation process is considered a necessary research direction to reduce

manual effort, improve test coverage, and support the improvement of software quality [10].

B. Why Existing LLM-based Methods Fail

The rapid development of Large Language Models (LLM) has opened up many new potentials for the problem of automated test code generation. Studies on program synthesis and code generation show that LLMs are capable of generating code from descriptions and input contexts [2], [3]. Models such as GPT-4, Code Llama, DeepSeek-Coder, and Qwen2.5-Coder have demonstrated outstanding capabilities in understanding source code, analyzing programming context, reasoning about program behavior, and generating code with a structure close to human-written code [6], [7], [11], [12]. Among them, GPT-4 shows strong general capability across many programming tasks; Code Llama is designed specifically for code; DeepSeek-Coder focuses on programming ability and source code understanding; and Qwen2.5-Coder demonstrates the ability to generate and process code across multiple programming languages. Thanks to these characteristics, LLMs can support unit test generation by reading the input class or method, reasoning about the main execution branches, creating test data, and writing corresponding assertions.

Compared with traditional search-based unit test generation tools such as EvoSuite, LLM-based methods have the advantage that the generated test code is usually more readable, closer to the testing style of developers, and has greater potential for maintainability [1], [8], [13]. Instead of only generating combinational or difficult-to-interpret test cases, LLMs can generate tests with clear method names, understandable Arrange-Act-Assert structures, and assertions that express the testing purpose. This helps LLM-generated tests become more suitable for real software development workflows, where developers not only need tests that can run but also tests that are readable, editable, and extensible.

However, when applied to real-world Java projects, current LLM-based test generation methods still have many limitations. First, many methods focus on methods or independent testing units [9], while real Java classes often contain constructors, fields, internal states, inheritance, dependency injection, and interactions with external components. Therefore, LLMs may not fully understand the class-level context and may generate tests that do not correctly reflect the actual behavior of the object under test [14], [15], [17]. Second, LLMs may generate test code with compilation errors or runtime errors, such as calling non-existent APIs, importing incorrect libraries, using incorrect data types, missing mock objects, or creating inappropriate assertions [4], [13], [16]. Third, current systems often lack a controlled automatic repair mechanism after generated tests contain errors, making the retry process costly, unstable, and difficult to integrate into practical software development pipelines [4], [16]. Therefore, although LLMs bring significant potential for automated unit test generation, a framework is still needed that can effectively handle class-level context, automatically repair generated tests, and evaluate test quality comprehensively.

C. Overview of ARROW

To address the above gaps, this study proposes ARROW an end-to-end framework for automatically generating, repairing, and evaluating class-level unit test suites for Java projects. ARROW supports the entire process from analyzing the input repository, configuring the test generation strategy, using LLMs to generate test code, compiling and executing tests, repairing based on feedback, to evaluating test quality using standard tools. The framework consists of four main stages: Strategy Configuration, Automated Test Generation, Adaptive Repair, and Test Suite Assessment**.**

D. Main Contributions

This study has three main contributions. First, ARROW proposes a repository-aware generation workflow based on a fine-tuned open-source LLM, capable of analyzing build tool, Java version, module context, testing framework, and existing test style to construct project-specific prompts [4], [16], [18]. Second, ARROW is empirically evaluated using compilation success, module-test pass rate, repair success, code coverage, mutation score, and test smell metrics. Compilation success represents the proportion of generated tests that compile successfully, while the module-test pass rate measures whether the tests can be executed successfully within the target module. Repair success indicates the framework's ability to correct initially failed tests. In addition, code coverage measures the extent of the source code exercised by the tests, mutation score evaluates their fault-detection capability, and test smell metrics identify structural and maintainability issues in the generated tests [13], [17].

Third, ARROW proposes an adaptive repair algorithm that uses execution feedback to classify failure states, track normalized error signatures and generated-code hashes, and apply checkpointing and rollback to avoid ineffective repair loops and limit regressions during the iterative repair process [13], [16], [17].

II. RELATED WORK

A. Evolution of Automated Unit Test Generation

Over the past five years, automated unit test generation has shifted from deep learning translation to Large Language Model (LLM)-driven pipelines. AthenaTest (2021) [21] pioneered this by framing test generation as a sequence-to-sequence translation task using a Transformer. However, by treating generation as an isolated method-level translation problem and ignoring repository-wide context (project hierarchies, dependencies, and build configurations), AthenaTest suffers from low compilation rates. To improve execution reliability, LIBRO (2023) [22] introduced LLM-based bug reproduction from bug reports; though demonstrating LLMs' semantic capabilities, LIBRO is specialized for reproducing reported bugs rather than generating general class-level coverage. Concurrently, A3Test (2024) [23] proposed assertion-augmented pre-training via domain adaptation to enhance assertion correctness, yet still neglects project-level configuration metadata (build tools, compiler versions, dependencies) required for compilation. Recently, Meta's TestGen-LLM (2024) [24] demonstrated industrial LLM-based test generation using verification filters; however, it only extends existing test classes (Kotlin), generates tests in a single-shot manner, and lacks adaptive repair to handle compilation or regression failures. In summary, existing approaches treat test generation in isolation, lacking a unified end-to-end workflow integrating repository analysis, generation, adaptive repair, and quality assessment.

B. Context-Aware Prompt Construction for LLM-based Test Generation

To ensure correctness, recent LLM-based frameworks utilize context-aware prompts to reduce hallucinations (e.g., invoking non-existent APIs) when only the class-under-test (CUT) is provided. ChatUniTest [25] extracts class-level context (inheritance, constructors, dependent signatures) to build prompts. However, operating solely at the source-code layer while neglecting repository-wide configurations (build tools, compiler versions, test framework specifications), ChatUniTest suffers from compilation failures due to library mismatches in practice. TestSpark [26], an IntelliJ plugin, integrates search-based testing with LLMs; though providing a developer-friendly workflow, it lacks automated repository-aware analysis, and its repair relies on a basic compilation feedback loop without advanced failure-state analysis. Critically, neither framework treats repository analysis as a structured, independent stage in the pipeline, failing to propagate configuration metadata to downstream repair and evaluation phases.

C. Feedback-Driven Validation and Adaptive Repair Loops

Since LLM-generated tests often contain syntax bugs, feedback-driven refinement is critical. ChatTester [27] uses a "validate-and-fix" cycle based on compiler feedback, but applies a fixed repair strategy - handling compilation, runtime, and assertion failures with the same generic prompt without failure-state classification. TestART [28] integrates a co-evolution approach, repairing compilation errors via templates and refining tests through coverage feedback; however, it lacks explicit failure-state classification, checkpointing, and rollback. Furthermore, existing frameworks (including ChatUniTest [25]) do not track systematic error signatures and code hashes to detect stagnation, nor do they support rollback during regression - gaps that an adaptive repair loop with error classification and state-aware control must address.

D. Test Quality Assessment and Comparative Benchmarking

Evaluating test quality requires multi-dimensional assessments (coverage, mutation testing, and test smell detection) rather than simple line coverage. AgoneTest [29] integrates PiTest, tsDetect, and JaCoCo to evaluate LLM-generated tests. However, existing evaluations are often applied in isolation on pre-selected datasets without conducting rigorous comparative benchmarks against traditional search-based software testing (SBST) tools (e.g., EvoSuite) and state-of-the-art LLM-based generators under identical project setups. Furthermore, most studies fail to assess the entire spectrum of test quality - specifically the combination of compilation success, execution pass rates, repair efficiency, and code maintainability - limiting objective assessment. Thus, there is a need for a systematic evaluation methodology that objectively measures generated tests across all critical quality dimensions and performs direct comparative benchmarks under identical repository configurations.

E. Research Gap Summary

The foregoing analysis reveals three concrete research gaps. Gap 1 (End-to-end integration): Existing frameworks treat test generation stages in isolation. No framework constructs a unified end-to-end pipeline that propagates information across repository analysis, context-aware generation, adaptive repair, and quality assessment as a coherent feedback loop, meaning information produced at each stage is rarely reused in subsequent steps. Gap 2

(Adaptive repair with state-aware control): Existing repair approaches lack intelligent control. They do not classify failure types to select repair strategies, fail to detect stagnation when regenerating identical erroneous code, and lack rollback mechanisms to recover from regression. Gap 3 (Comprehensive and Comparative Evaluation): Existing evaluations are limited to simple coverage metrics or isolated test suites. They lack a unified, multi-dimensional benchmark (compilation success, execution pass rate, mutation score, test smells) directly comparing LLM-generated tests against traditional search-based tools and alternative LLM frameworks under identical settings. This motivates the need for an integrated framework addressing all three gaps simultaneously, as detailed in the next section.

III. PROPOSED METHOD

We propose ARROW an end-to-end framework for generating and multidimensionally evaluating the quality of class-level unit test suites in real-world Java projects. As illustrated in Figure 1, the system consists of four tightly integrated modules: (1) Repository-aware, (2) LLM-Based Test Generation, (3) Adaptive Repair Strategy, and (4) Automated Test Suite Assessment.

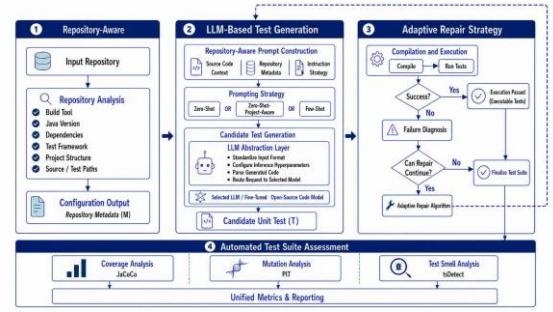

Figure 1. Overview of the ARROW System Architecture.

A. *Repository-aware*

This module preprocesses the target project through three consecutive stages: Select Project, Analyze, and Config. First, in the Select Project stage, users select a set of sample projects with diverse levels of complexity, ranging from small libraries to multi-module projects with complex inter-class dependencies, to ensure the objectivity of the evaluation. Next, in the Analyze stage, the framework performs static analysis, including code cleaning, format normalization, syntax parsing to extract packages, classes, constructors, and methods, cyclomatic complexity calculation, and dependency graph construction among modules. Finally, in the Config stage, users configure environmental parameters such as the JDK version, Maven/Gradle build tool, JUnit/Mockito testing framework, dependencies, and retry budget for the automatic repair of failed unit tests by the LLM. All parameters are packaged into a configuration file and combined with a prompt template for use in subsequent stages.

Project information and execution parameters are formalized using the function f:

$$M = f(R, Y)$$

where

R is the raw project repository

Y is the pipeline.yaml configuration file

f is the function that analyzes, normalizes, and combines the information.

The metadata set M is defined as follows:

M = (java_version, build_tool, dependencies, test_framework, source_classes, complexity_map, existing_tests, run_config, output_config, report_config, input_config, repo_config, cleanup_config, build_config, llm_config, prompt_config, experiment_config, adaptive_repair_config, metrics_config).

In this set, java_version, build_tool, dependencies, test_framework, source_classes, complexity_map, and existing_tests are metadata extracted from the repository. The remaining components are configuration groups declared in the pipeline.yaml file.

The set M contains all the parameters described above; however, most projects mainly focus on adjusting several major configuration groups. build_config defines the supported build tools, JDK versions and paths, maximum test execution time, and how Maven handles multi-module projects. llm_config specifies the provider, model, API endpoint, environment variable containing the API key, temperature, num_ctx, and max_tokens. input_config and experiment_config determine the projects to be executed, the number of samples per project, the models, and the prompt strategies used. adaptive_repair_config enables or disables automatic repair, configures the retry budget, detects repeated errors or lack of progress, switches repair strategies, and rolls back to the best candidate. Finally, metrics_config controls the collection of code coverage, mutation testing, and test smells. These configurations allow the pipeline to adapt to each project, execution environment, and LLM without modifying the core logic of the framework.

B. *LLM-Based Test Generation*

To improve the compatibility of generated unit tests with the target project, ARROW uses Prompt Creator to construct repository-aware prompts from the extracted metadata M, as shown in the Prompt

Strategy block of Module 2 in Fig. 1. The framework supports three prompt construction strategies:

(1) zero_shot, which uses the focal class and basic contextual information to generate tests

(2) few_shot, which adds high-quality and well-structured sample test cases to guide the generation process

(3) zero_shot_project_aware, which combines the focal class with project metadata, build tools, dependencies/APIs, and existing test styles. During generation, the prompt requires the model to use the correct build tool; use only APIs, methods, and imports available in the Context JSON; prioritize imports and assertions compatible with the project instead of modern APIs that may not exist; and imitate the existing test style. These constraints help significantly reduce calls to nonexistent APIs and improve the compilation and execution capability of generated unit tests.

The Unit Test Candidate generation process is formalized through the function G as follows:

$$G: (P, M, \theta) \rightarrow T$$

where

T is the generated unit-test candidate.

G is the code generation function of the LLM-Based Test Generation layer, shown in Module 2 of Fig. 1. It receives the prompt, project metadata, and inference parameters to generate unit test code.

P is the constructed prompt. Additional prompt templates are available at: https://github.com/dungng2808/ARROW-paper/tree/main/Sample%20Prompting;

M represents the project metadata and configuration described in Section 3.1

θ is the inference parameter set declared in the llm.agents group of the pipeline.yaml file:

At this stage, the LLM-Based Test Generation layer, shown in Module 2 of Fig. 1, receives P, M, and θ, normalizes the requirements, and routes them to the selected model to generate a Unit Test Candidate.

C. Adaptive Repair Strategy

After a Unit Test Candidate is created during the Automated Test Generation stage, the framework activates the Adaptive Repair Algorithm shown in Figure 2 to validate and automatically repair the unit test before transferring it to the quality assessment stage. First, the candidate is integrated into the target Java project and validated using Maven or Gradle in the configured environment, including the JDK version, build tool, testing framework, dependencies, and source-code paths. The validation process checks compilation and execution to identify syntax, import, dependency, runtime, or assertion errors. If the unit test executes successfully, the candidate is confirmed as valid and stored as the finalized test suite. Otherwise, the framework classifies the failure state based on compiler logs, stack traces, and execution results to determine the failure type that requires repair.

After each failure, the framework evaluates the level of progress by comparing the error state, normalized error signature, and source-code content with previous attempts. This mechanism detects cases in which the LLM reproduces the same error, generates duplicate code, or fails to improve the candidate. Based on the error type and repair history, the framework selects an appropriate repair strategy or switches to another strategy, then sends the error information and diagnostic details back to Prompt Creator so that the LLM can generate a repaired version. The new candidate is subsequently recompiled and re-executed. If the result is better, the version is stored as the new finalized test suite. If the error becomes more severe or the repair process shows no progress, the framework rolls back to the best previous candidate. The cycle of generation, validation, failure classification, repetition detection, strategy selection, repair, and re-validation continues until the unit test executes successfully or reaches the retry budget configured in the system.

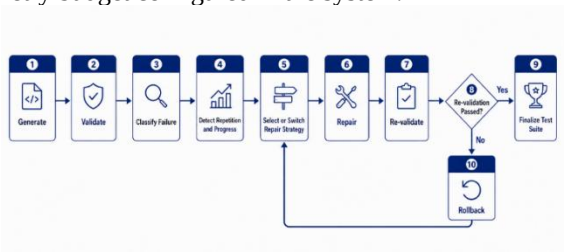

Figure 2. Adaptive Repair Algorithm.

D. Automated Test Suite Assessment

After the unit tests achieve the Pass state and complete the validation stage, ARROW evaluates their quality to measure the effectiveness and maintainability of the generated test suite. The quality of test suite T is represented by the following metric set:

$$Q(T) = \{C_{\text{cov}}, S_{\text{mut}}, D_{\text{smell}}\}$$

where Q is the quality assessment function of unit test suite T. C_{cov}, S_{mut}, and D_{smell} represent code coverage, mutation score, and test smells, respectively. These metrics are automatically collected using JaCoCo, PIT/PiTest, and tsDetect. The results are aggregated in Test Suite Assessment, enabling an objective comparison of the effectiveness of ARROW's prompting strategies, code-generation

models, and test-generation configurations on the same dataset.

IV. EXPERIMENTS AND RESULTS

This section presents the experimental design used to evaluate the three main components of ARROW: repository-aware prompt construction, adaptive repair, and multidimensional quality assessment of generated unit tests. The experiments are organized around three research questions:

RQ1: Does repository-aware prompt construction improve the compilation and execution success of generated unit tests compared with conventional prompting strategies?

RQ2: Does the Adaptive Repair mechanism improve the success rate of unit-test repair while reducing the number of repair attempts and repair time compared with a non-adaptive fixed repair mechanism?

RQ3: Can ARROW assess and distinguish the quality of unit tests generated by different LLMs and prompting strategies in terms of code coverage, fault-detection capability, and test-code quality?

The three research questions correspond to the main stages of the experimental workflow. RQ1 evaluates tests immediately after initial generation, RQ2 evaluates the repair of unsuccessful candidates, and RQ3 assesses the quality of valid tests after generation and repair.

A. Experimental Subjects and Dataset

The experiments use Classes2Test because it directly supports class-level unit-test generation. Each sample maps a focal class to its corresponding human-written test and provides the repository information and source paths required to reconstruct the project context. Its coverage of real-world Java repositories enables ARROW to be evaluated across diverse project structures, build tools, testing frameworks, and dependency configurations [29].

A valid candidate pool was constructed using the following criteria: the repository could be cloned and checked out at the specified commit; the focal class existed at the dataset path; the containing module was identifiable as Maven or Gradle; the testing framework, such as JUnit 4, JUnit 5, or TestNG, could be determined; the focal class was not a pure interface, generated source, or test class; and the project could be configured with a compatible JDK version.

After filtering, 200 class-level samples were randomly selected from the valid candidate pool. Each repository contributed at most one sample to reduce the influence of large repositories. The final benchmark therefore contained 200 samples from 200 distinct repositories. This scale balances project diversity with the cost of test generation, repair, coverage analysis, and mutation testing, following the representativeness-tractability principle used by AgoneTest [29]. The ARROW replication package provides the 200 sample identifiers, while detailed metadata can be retrieved from the original Classes2Test dataset.

Each sample is uniquely identified by the tuple:

$$s_i = \langle \text{repository\_id}, \text{commit\_hash}, \text{focal\_class} \rangle$$

where repository_id identifies the project, commit_hash specifies the source-code revision, and focal_class denotes the class for which unit tests are generated.

B. Experimental Environment and Configuration

Each experiment was executed in an isolated workspace. For every sample-model-prompt combination, ARROW created a separate repository copy to prevent generated tests, build artifacts, or repair results from affecting other experiments. This design also ensured that every run could be traced through its corresponding logs, artifacts, and reports.

All experiments were executed under a consistent software environment. The pipeline used Python 3.12.10 and LiteLLM 1.91.0 as the unified inference interface [20]. The fine-tuned Qwen model was imported and served locally through Ollama [19]. For Java projects, ARROW selected the JDK version from repository metadata or build configuration, with JDK 8, 11, 17, and 21 available. Maven 3.9.16 was used for Maven projects, while Gradle projects preferentially used the repository-provided Gradle Wrapper. Coverage, mutation score, and test smells were measured using JaCoCo 0.8.12, PIT 1.17.4, and Test Smell Detector (tsDetect), respectively, consistent with multidimensional assessment practices in prior work [29].

Two model groups were used to represent different deployment scenarios. GPT-4 was selected as a strong proprietary baseline because of its reported performance on code-generation benchmarks, including HumanEval and MBPP [2], [3], [11]. Qwen2.5-Coder-32B represented an open-source, code-specialized model that can be deployed through self-hosted inference [7]. The 32B variant was selected as a higher-capacity open-source baseline and was served in quantized form through Ollama for the experimental deployment.

In this study, Qwen2.5-Coder-32B was fine-tuned on approximately 20,000 Java class-test pairs extracted from Classes2Test. Each pair was converted into an instruction sample in which the input contained the focal class, focal method, and related

context, while the output contained the corresponding unit test. Fine-tuning was intended to adapt the model to class-level unit-test generation and common Java testing conventions, including JUnit, Mockito, imports, assertions, and test structure. The dataset size was selected as a practical trade-off between data diversity, training cost, and available computing resources. This scale is comparable to WaveCoder's CodeOcean dataset, which contains 19,915 instruction instances across four code-related tasks [30], while Magicoder provides a larger reference point by using 75,000 synthetic code instructions [33]. More generally, LIMA shows that 1,000 carefully curated examples can be sufficient for learning response formats and generalizing to unseen tasks [31], and low-data instruction-tuning research suggests that task-specific adaptation may remain effective with a small, relevant subset of the available data [32]. Because these studies use different models and tasks, this study does not claim that 20,000 samples are optimal for Java unit-test generation.

To reduce the risk of direct sample-level data leakage, the sample identifiers used in the 200-sample evaluation set were removed from the fine-tuning dataset. This prevents the exact evaluation samples from appearing in training; however, because the split was performed at the sample level rather than the repository level, possible overlap through other samples from the same repositories remains a limitation.

All model-prompt combinations were evaluated on the same samples and under the same experimental conditions. ARROW compares a proprietary model with a fine-tuned open model while also evaluating repository-aware prompting and adaptive repair. The purpose is to examine whether ARROW behaves consistently across different model types rather than to establish a universally superior model.

The number of initial generations in RQ1 is defined as:

$$E_{\mathrm{RQ1}} = M_s \times L \times P \times r$$

where M_s is the number of samples, L is the number of models, P = 3 is the number of prompting strategies, and r is the number of repetitions.

C. General Experimental Procedure

For each sample, ARROW analyzed the repository to identify the target module, Java version, build tool, testing framework, dependencies, and public API of the focal class. The selected strategy was then used to construct the prompt, and the LLM generated a Java test through LiteLLM [20].

The generated test was format-checked, written to the appropriate module, compiled, and executed in isolation before the complete module test suite was run. A test was counted as executed only when Maven Surefire, Maven Failsafe, or Gradle confirmed that at least one test method from the generated class had run.

Adaptive Repair was disabled in RQ1 to measure the quality of the initial generation directly. In RQ2, failed candidates were processed by either Fixed Repair or Adaptive Repair. In RQ3, valid tests were assessed using JaCoCo, PIT, and tsDetect [29].

D. RQ1 - Effect of Zero-Shot Project-Aware Prompting

1) Experimental Design

Three strategies were compared: Zero-shot, which provided only the focal class and a test-generation instruction; Few-shot, which added source-test examples from other repositories; and Zero-shot-project-aware, which additionally supplied the Java version, build tool, testing framework, dependencies, module, public API, and existing test style. Few-shot examples did not contain the focal class or the human-written test of the target sample, thereby reducing data-leakage risk.

Adaptive Repair was disabled throughout RQ1. Because the same samples and execution conditions were used for all three strategies, the prompting strategy was the primary independent variable.

2) Evaluation Metrics

Compilation Success Rate

Compilation Success Rate measures the proportion of generated tests that compile successfully in the target module:

$$\mathrm{CSR_p} = \frac{N_{\mathrm{compiled},p}}{N_{\mathrm{generated},p}} \times 100\%$$

where $N_{\mathrm{compiled,p}}$ is the number of tests compiled successfully under prompting strategy p, and $N_{\mathrm{generated,p}}$ is the total number of generated tests under that strategy. Since each strategy was evaluated on 200 samples using two models, $N_{\mathrm{generated},p} = 400$.

Test Execution Success Rate

Test Execution Success Rate measures the proportion of generated tests detected by the test framework and executed with at least one test method:

$$\mathrm{ESR_p} = \frac{N_{\mathrm{executed},p}}{N_{\mathrm{generated},p}} \times 100\%$$

where $N_{\mathrm{executed},p}$ is the number of generated tests that are discovered by the test framework and execute at least one test method under prompting strategy p.

A test was considered executed when the generated test class was discovered and at least one of

its test methods ran. A test with an assertion failure was still counted as executed, but it was not counted as passed.

Target Test Pass Rate was collected as an additional metric:

$$\mathrm{TPR_p} = \frac{N_{\text{target\_passed},p}}{N_{\text{generated},p}} \times 100\%$$

Where $N_{\text{target\_passed},p}$ is the number of generated tests whose target test passes under prompting strategy p.

3) RQ1 Results

Table I reports the compilation, execution, and target-pass results for Zero-shot, Few-shot, and Zero-shot-project-aware prompting. Each strategy includes 400 generations, corresponding to 200 samples evaluated with GPT-4 and 200 samples evaluated with Qwen2.5-Coder-32B. Adaptive Repair was disabled so that the results reflect the quality of the initial generation.

TABLE I. COMPILATION AND EXECUTION RESULTS BY PROMPTING STRATEGY

Prompt strategy	**Total generations**	**CSR, n (%)**	**ESR, n (%)**	**TPR, n (%)**
Zero-shot	400	250 (62.50%)	222 (55.50%)	98 (24.50%)
Few-shot	400	270 (67.50%)	237 (59.25%)	109 (27.25%)
Zero-shot-project-aware	400	323 (80.75%)	271 (67.75%)	144 (36.00%)

N=total generations; CSR=Compilation Success Rate; ESR=Test Execution Success Rate; TPR=Target Test Pass Rate.

Zero-shot-project-aware achieved the best results on all three metrics. Compared with Zero-shot, it improved Compilation Success Rate by 18.25 percentage points, Test Execution Success Rate by 12.25 percentage points, and Target Test Pass Rate by 11.50 percentage points.

Compared with Few-shot, the corresponding improvements were 13.25, 8.50, and 8.75 percentage points. These results indicate that repository information helps the models avoid incompatible imports, dependencies, API calls, and test configurations.

Among executed tests, the target-pass rates for Zero-shot, Few-shot, and Zero-shot-project-aware were 44.14%, 45.99%, and 53.14%, respectively. Repository context therefore improved not only compilation compatibility but also the likelihood that an executed generated test behaved correctly.

4) Answer to RQ1

Zero-shot-project-aware prompting substantially improved the compilation, execution, and target-pass rates of generated unit tests compared with Zero-shot and Few-shot. Repository context therefore had a positive effect on the quality of the initial test generation.

E. RQ2 - Effectiveness of the Adaptive Repair Mechanism

RQ2 evaluates whether ARROW's Adaptive Repair mechanism can repair more failed generated tests while reducing repair attempts and repair time compared with a fixed, non-adaptive mechanism. Unlike RQ1, which evaluates only the initial generation, RQ2 focuses on candidates that did not reach a successful state after initial verification.

1) Experimental Design

The candidate set for RQ2 is defined as follows:

$$F = \{\, T_i \mid \mathrm{state}(T_i) \neq \mathrm{MODULE\_TESTS\_PASSED} \,\}$$

Here, T_i denotes the i-th generated test from RQ1. A candidate reaches MODULE_TESTS_PASSED only when the generated target test passes and the complete module test suite shows no regression. Therefore, F contains candidates with compilation errors, failed test discovery, runtime errors, assertion failures, or regressions in existing human-written tests.

From the failed candidates across the six model-prompt combinations in RQ1, 120 candidates were selected for the controlled comparison. The same candidates were copied into independent workspaces and evaluated under three mechanisms. No Repair retained the initial candidate without modification. Fixed Repair used one fixed repair prompt in every round and always continued from the latest candidate. Adaptive Repair classified failure states, tracked normalized error signatures and generated-code hashes, and applied checkpointing, rollback, and repair-strategy switching when repetition, lack of progress, or regression was detected.

For a fair comparison, Fixed Repair and Adaptive Repair used the same initial candidate, the model that originally generated that candidate, inference parameters, retry budget, build timeout, and repository state. Both mechanisms were allowed to modify only the generated test; the focal class, dependencies, build configuration, and human-written tests remained unchanged. Repair terminated

when the candidate reached MODULE_TESTS_PASSED or when the retry budget was exhausted, a build timeout occurred, the same code or error signature reappeared, no repair strategy remained, or a tool error occurred.

2) Evaluation Metrics

Final Compilation Success Rate measures the proportion of initially failed candidates that compile at the end of the repair process:

$$\mathrm{FCSR} = \frac{N_{\mathrm{final,compiled}}}{N_{\mathrm{initial,failed}}} \times 100\%$$

where $N_{\mathrm{final,compiled}}$ is the number of candidates that compile in their final state, and $N_{\mathrm{initial,failed}}$ is the total number of initially failed candidates.

Final Test Pass Rate measures the proportion of initially failed candidates whose generated target test passes after repair.

$$\mathrm{FTPR} = \frac{N_{\mathrm{final,passed}}}{N_{\mathrm{initial,failed}}} \times 100\%$$

Where $N_{\mathrm{final,passed}}$ is the number of initially failed candidates whose generated target test passes after repair.

Repair Success Rate considers a repair successful only when the generated target test passes and the complete module test suite introduces no new regression.

$$\mathrm{RSR} = \frac{N_{\mathrm{successfully,repaired}}}{N_{\mathrm{repair,attempted}}} \times 100\%$$

Where $N_{\mathrm{successfully,repaired}}$ is the number of candidates whose generated target test passes without introducing a regression in the complete module test suite, and $N_{\mathrm{repair,attempted}}$is the number of initially failed candidates submitted to the repair mechanism.

Repair Attempts (RA) is the number of LLM repair calls made before the test succeeds or a stopping condition is reached. Because attempt counts are typically non-normal, they are reported as median and interquartile range, median [IQR].

Repair Time (RT) is measured from the beginning of the first repair round until the candidate is successfully repaired or the process stops because of retry-budget exhaustion, strategy exhaustion, timeout, or tool failure.

3) RQ2 Results

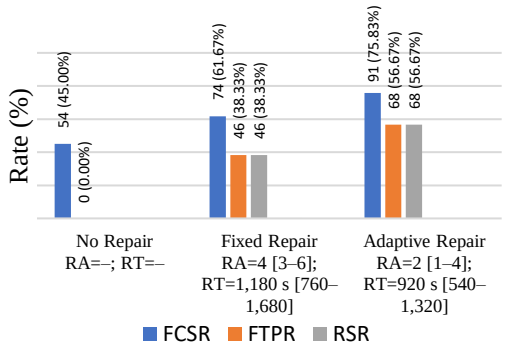

Figure 3. Effectiveness and efficiency of the evaluated repair mechanisms.

Figure 3 compares the effectiveness and efficiency of the evaluated repair mechanisms. Under the No Repair baseline, 54 of the 120 candidates compiled successfully, corresponding to an FCSR of 45.00%, but none reached the final target-test pass state. Fixed Repair increased the FCSR to 61.67%, with 74 candidates compiling successfully, and enabled 46 candidates, or 38.33%, to pass the generated target test. Adaptive Repair performed better, achieving an FCSR of 75.83% and an FTPR of 56.67%, corresponding to 91 and 68 candidates, respectively.

Compared with Fixed Repair, Adaptive Repair improved the final compilation rate by 14.16 percentage points and the final target-test pass rate by 18.34 percentage points. For both repair mechanisms, FTPR and RSR had identical values because every candidate whose generated target test passed also completed the module test suite without introducing regressions.

Adaptive Repair also reduced the median number of repair attempts from 4 to 2, representing a 50.00% reduction, and reduced the median repair time from 1,180 seconds to 920 seconds, a decrease of approximately 22.03%. The reduction in repair time was smaller than the reduction in repair attempts because compilation and full module-test execution still accounted for a substantial proportion of the overall repair cost.

4) Answer to RQ2

Adaptive Repair achieved a 56.67% repair success rate, 18.34 percentage points above Fixed Repair, while reducing the median number of repair attempts from 4 to 2 and the median repair time from 1,180 to 920 seconds. Failure classification, repetition detection, checkpointing, rollback, and strategy switching therefore improved repair efficiency and limited regression.

F. RQ3 - Quality of Automatically Generated Tests

The 120 candidates in RQ2 were used only for the controlled comparison between Fixed Repair and Adaptive Repair. After that comparison, Adaptive Repair was applied to all RQ1 candidates that had not reached MODULE_TESTS_PASSED. Tests that compiled, executed, passed the target test, and introduced no module regression were forwarded to RQ3 for quality assessment.

RQ3 evaluates three dimensions: code coverage, fault-detection capability through mutation testing, and structural test-code quality through test-smell detection. Together, these metrics provide a broader assessment than coverage alone [13], [17], [29]. Values were computed on the valid-test set of each model-prompt combination.

1) Coverage and Mutation Results

JaCoCo measured instruction, line, branch, and method coverage, while PIT measured Mutation Score. For each configuration, metrics were computed for each valid focal class and then averaged across the valid classes, following the multidimensional assessment approach used in prior work [29].

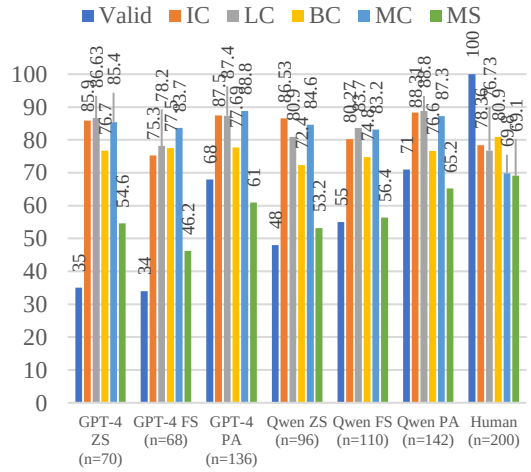

Figure 4. Valid-test rate, coverage, and mutation results across model–prompt configurations

G4=GPT-4; QW=Qwen2.5-Coder-32B; ZS=Zero-shot; FS=Few-shot; PA=Zero-shot-project-aware. Valid-test counts for G4-ZS, G4-FS, G4-PA, QW-ZS, QW-FS, QW-PA, and Human are 70, 68, 136, 96, 110, 142, and 200, respectively. Valid=Percentage of valid tests; IC=Instruction coverage; LC=Line coverage; BC=Branch coverage; MC=Method coverage; MS=Mutation score. All reported metric values are percentages, except the valid-test counts. Coverage and mutation values were computed only on the valid-test set of each configuration.

Figure 4 illustrates the quality of valid tests in terms of code coverage and mutation score across all model–prompt configurations. Among the generated-test settings, zero-shot-project-aware consistently yielded the strongest results on most metrics. In the case of GPT-4, it improved the Mutation Score to 61.00%, outperforming both Zero-shot and Few-shot. For Qwen2.5-Coder-32B, the same strategy achieved the best generated-test performance overall, with 88.31% instruction coverage, 88.80% line coverage, and a 65.20% Mutation Score.

In contrast, human-written tests still achieved the highest Mutation Score (69.10%), suggesting that broader coverage alone does not guarantee superior fault-detection effectiveness. Since the number of valid tests varies across configurations, these results are best interpreted as descriptive indicators of quality trends rather than a strict ranking of models.

2) Test-Smell Results Using tsDetect

tsDetect was used to provide structural indicators related to test maintainability [29]. Smell Density was calculated as the total number of detected smell instances divided by the number of valid tests in each group; lower values indicate fewer structural issues. Smell-free Tests reports the number and percentage of tests for which tsDetect detected no smell.

TABLE II. TEST-SMELL RESULTS FOR VALID TESTS

Model	Prompt	Valid tests	Smell Density	Smell-free Tests
GPT-4	Zero-shot	70	0.42	43 (61.43%)
GPT-4	Few-shot	68	0.48	39 (57.35%)
GPT-4	Zero-shot-project-aware	136	0.31	94 (69.12%)
Qwen2.5-Coder-32B	Zero-shot	96	0.55	52 (54.17%)
Qwen2.5-Coder-32B	Few-shot	110	0.46	68 (61.82%)
Qwen2.5-Coder-32B	Zero-shot-project-aware	142	0.32	96 (67.61%)
Human-written tests	-	200	0.18	168 (84.00%)

Zero-shot-project-aware again produced the best generated-test results. GPT-4 achieved a Smell Density of 0.31 and a smell-free rate of 69.12%, while Qwen2.5-Coder-32B achieved 0.32 and 67.61%, respectively. Human-written tests still had the lowest Smell Density and the highest smell-free rate. Repository context therefore improved the structure and maintainability indicators of generated tests, although a gap remained relative to human-written tests.

3) Answer to RQ3

The results demonstrate that ARROW can assess and distinguish generated-test quality across multiple dimensions. Zero-shot-project-aware generally produced better coverage, Mutation Score, and test-smell results than Zero-shot and Few-shot. However, human-written tests retained stronger fault-detection capability and better structural quality.

Overall, Zero-shot-project-aware improved initial test generation, Adaptive Repair repaired failures more effectively than Fixed Repair with fewer attempts and less time, and the combined use of coverage, mutation testing, and test-smell analysis enabled a multidimensional evaluation of test quality. Nevertheless, the generated tests did not fully replace human-written tests.

V. CONCLUSION AND FUTURE WORK

This study proposed **ARROW**, an end-to-end framework for repository analysis, class-level unit test generation, automated repair, and test quality assessment using LLMs. The results show that repository-aware prompting improves compilation success, while Adaptive Repair increases the repair success rate and reduces repair costs.

This study has several limitations. ARROW was evaluated on 200 class-level samples using GPT-4 and Qwen2.5-Coder-32B; therefore, its generalizability to other repositories, models, and industrial projects requires further validation. In addition, coverage, mutation score, and test smell metrics were measured only on tests that passed the validation stage, while semantic correctness, flaky tests, inference costs, and the individual contribution of each framework component were not separately evaluated. These limitations provide directions for future work, including larger benchmarks, ablation studies, and the integration of ARROW into real-world CI/CD pipelines.

Future work will extend ARROW to more programming languages, repositories, and LLMs, while improving assertion generation, test discovery, dependency handling, and CI/CD integration.

REFERENCES

[1] L. Chen, Y. Zhang, Y. Liu, J. Yang, M. Yang, and Y. Jiang, "An empirical study on automated test generation tools for Java: Effectiveness and challenges," J. Comput. Sci. Technol., vol. 39, no. 4, pp. 890-910, 2024.

[2] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, "Program synthesis with large language models," arXiv preprint arXiv:2108.07732, 2021.

[3] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, et al., "Evaluating large language models trained on code," arXiv preprint arXiv:2107.03374, 2021.

[4] Y. Chen, Z. Hu, C. Zhi, J. Han, S. Deng, and J. Yin, "ChatUniTest: A framework for LLM-based test generation," in Proc. ESEC/FSE '24, 2024, pp. 572-576.

[5] V. Garousi and E. van Veenendaal, "Test Maturity Model integration (TMMi): Trends of worldwide test maturity and certifications," IEEE Software, vol. 39, no. 2, pp. 71-79, Mar.-Apr. 2022.

[6] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. K. Li, F. Luo, Y. Xiong, and W. Liang, "DeepSeek-Coder: When the large language model

meets programming - The rise of code intelligence," arXiv preprint arXiv:2401.14196, 2024.

[7] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu, et al., "Qwen2.5-Coder technical report," arXiv preprint arXiv:2409.12186, 2024.

[8] C. Lemieux, J. P. Inala, S. K. Qi, T. Sber, S. K. Lahiri, and S. Sen, "CodaMosa: Escaping coverage plateaus in test generation with pre-trained large language models," in Proc. 45th IEEE/ACM Int. Conf. on Software Engineering (ICSE '23), Melbourne, Australia, 2023, pp. 919-931.

[9] S. Lukasczyk and G. Fraser, "Pynguin: Automated unit test generation for Python," in Proc. 44th IEEE/ACM Int. Conf. on Software Engineering: Companion (ICSE-Companion '22), Pittsburgh, PA, USA, 2022, pp. 168-172.

[10] P. Nie, R. Banerjee, J. J. Li, R. J. Mooney, and M. Gligoric, "Learning deep semantics for test completion," in Proc. 45th IEEE/ACM Int. Conf. on Software Engineering (ICSE '23), Melbourne, Australia, 2023, pp. 2111-2123.

[11] OpenAI, "GPT-4 technical report," arXiv preprint arXiv:2303.08774, 2023.

[12] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez, J. Rapin, A. Kozhevnikov, I. Evtimov, J. Bitton, M. Bhatt, C. C. Ferrer, A. Grattafiori, W. Xiong, A. Defossez, J. Copet, F. Azhar, H. Touvron, L. Martin, N. Usunier, T. Scialom, and G. Synnaeve, "Code Llama: Open foundation models for code," arXiv preprint arXiv:2308.12950, 2023.

[13] M. Schafer, S. Nadi, A. Eghbali, and F. Tip, "An empirical evaluation of using large language models for automated unit test generation," IEEE Trans. Softw. Eng., vol. 50, no. 1, pp. 85-105, Jan. 2024.

[14] Y. Tang, Z. Liu, Z. Zhou, and X. Luo, "ChatGPT vs SBST: A comparative assessment of unit test suite generation," IEEE Trans. Softw. Eng., vol. 50, no. 5, pp. 1261-1278, May 2024.

[15] R. Tufano, D. Drain, A. Svyatkovskiy, S. K. Deng, and N. Sundaresan, "Unit test case generation with transformers and focal context," arXiv preprint arXiv:2009.05617, 2021.

[16] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, "Software testing with large language models: Survey, landscape, and vision," IEEE Trans. Softw. Eng., vol. 50, no. 4, pp. 911-936, Apr. 2024.

[17] Z. Yuan, Y. Liu, S. Wang, Z. Cai, L. Xue, Y. Deng, J. Chen, and X. Wang, "No more manual tests? Evaluating and improving ChatGPT for unit test generation," arXiv preprint arXiv:2305.04207, 2024.

[18] D. Zan, B. Chen, F. Zhang, D. Lu, B. Wu, B. Guan, Y. Wang, and J.-G. Lou, "Large language models meet NL2Code: A survey," in Proc. 61st Annual Meeting of the Association for Computational Linguistics (ACL '23), Toronto, Canada, 2023, pp. 7443-7468.

[19] Ollama, "Importing a Model," Ollama Documentation, 2026. Accessed: Jul. 11, 2026.

[20] LiteLLM, "Getting Started," LiteLLM Documentation, 2026. Accessed: Jul. 11, 2026.

[21] M. Tufano, D. Drain, A. Svyatkovskiy, S. K. Deng, and N. Sundaresan, "Unit test case generation with transformers and focal context," arXiv preprint arXiv:2009.05617, 2021.

[22] S. Kang, J. Yoon, and S. Yoo, "Large Language Models are Few-shot Testers: Exploring LLM-based General Bug Reproduction," in Proc. 45th IEEE/ACM Int. Conf. on Software Engineering (ICSE), 2023.

[23] S. Alagarsamy, C. Tantithamthavorn, and A. Aleti, "A3Test: Assertion-augmented automated test case generation," Information and Software Technology, vol. 176, p. 107565, 2024.

[24] N. Alshahwan et al., "Automated Unit Test Improvement using Large Language Models at Meta," in Proceedings of the ACM on Software Engineering (FSE), 2024.

[25] Y. Chen et al., "ChatUniTest: A framework for LLM-based test generation," in Companion Proc. 32nd ACM Int. Conf. on the Foundations of Software Engineering (FSE Companion), 2024, arXiv:2305.04764.

[26] A. Sapozhnikov, M. Olsthoorn, A. Panichella, V. Kovalenko, and P. Derakhshanfar, "TestSpark: IntelliJ IDEA's Ultimate Test Generation Companion," in Proc. 46th IEEE/ACM Int. Conf. on Software Engineering: Companion (ICSE Companion), 2024.

[27] Z. Yuan et al., "No More Manual Tests? Evaluating and Improving ChatGPT for Unit Test Generation," Proceedings of the ACM on Software Engineering (FSE), vol. 1, art. 76, 2024.

[28] S. Gu et al., "TestART: Improving LLM-based unit test via co-evolution of automated generation and repair iteration," arXiv preprint arXiv:2408.03095, 2024.

[29] A. Lops, M. Trizio, F. Narducci, C. Bartolini, and A. Ragone, "AgoneTest: Automated creation and assessment of Unit tests leveraging Large Language Models," in Proc. 39th IEEE/ACM Int. Conf. on Automated Software Engineering (ASE), 2024.

[30] Z. Yu, X. Zhang, N. Shang, Y. Huang, C. Xu, Y. Zhao, W. Hu, and Q. Yin, "WaveCoder: Widespread and versatile enhancement for code large language models by instruction tuning," in Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL), Bangkok, Thailand, 2024, pp. 5140-5153.

[31] C. Zhou, P. Liu, P. Xu, S. Iyer, J. Sun, Y. Mao, X. Ma, A. Efrat, P. Yu, L. Yu, S. Zhang, G. Ghosh, M. Lewis, L. Zettlemoyer, and O. Levy, "LIMA: Less is more for alignment," in Advances in Neural Information Processing Systems 36 (NeurIPS 2023), 2023, pp. 55006-55021.

[32] H. Chen, Y. Zhang, Q. Zhang, H. Yang, X. Hu, X. Ma, Y. Yanggong, and J. Zhao, "Maybe only 0.5% data is needed: A preliminary exploration of low training data instruction tuning," arXiv preprint arXiv:2305.09246, 2023.

[33] Y. Wei, Z. Wang, J. Liu, Y. Ding, and L. Zhang, "Magicoder: Empowering code generation with OSS-Instruct," in Proc. 41st Int. Conf. Machine Learning (ICML), vol. 235, 2024, pp. 52632-52657.